

DSA — RECURSION: PRINTING NUMBERS

1. MODULE INTRODUCTION

Recursion is one of the most important concepts in DSA.

Many problems can be solved by repeatedly solving a **smaller version of the same problem**.

For example, suppose we want to print:

5
4
3
2
1

Instead of thinking about all five numbers at once, recursion allows us to think:

Print 5

↓

Now solve the smaller problem: print 4 to 1

Then:

Print 4

↓

Now solve the smaller problem: print 3 to 1

And so on.

Eventually:

$5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

At 0, we stop.

This simple idea is the foundation of recursion.

2. INTUITION OF RECURSION

2.1 What Does Recursion Actually Mean?

Recursion is a technique in which a function calls itself to solve a smaller version of the same problem.

For example:

```
def fun(num):  
    fun(num - 1)
```

Here, `fun()` is calling itself.

That is the basic idea of recursion.

But simply calling a function again is not enough.

A correct recursive solution needs to answer three questions:

1. When should I stop?
2. How should I make the problem smaller?
3. What should I do at each step?

These correspond to:

Base Case

+

Progress

+

Recursive Call

3. THE THREE BUILDING BLOCKS OF RECURSION

Every beginner recursion problem should be analyzed using these three parts.

3.1 Base Case

The **base case** is the condition that stops recursion.

Example:

```
if num == 0:  
    return
```

It means:

When `num` becomes `0`, stop calling the function.

Without a base case, the function has no instruction to stop.

3.2 Recursive Call

A recursive call is when the function calls itself.

Example:

```
fun(num - 1)
```

The function:

```
fun(5)
```

calls:

```
fun(4)
```

which calls:

```
fun(3)
```

and so on.

3.3 Progress Toward the Base Case

The recursive call must make progress toward the base case.

Suppose our base case is:

```
if num == 0:  
    return
```

Then this is correct:

```
fun(num - 1)
```

because:

5 → 4 → 3 → 2 → 1 → 0

But this is incorrect:

```
fun(num + 1)
```

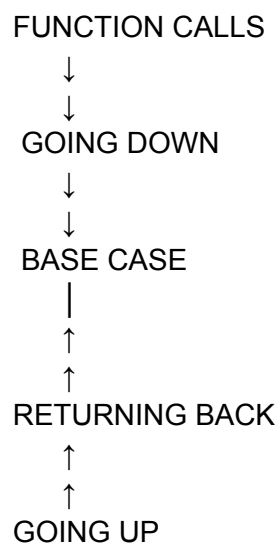
because:

5 → 6 → 7 → 8 → 9 → ...

The function is moving away from the stopping condition.

4. RECURSION MENTAL MODEL

Think of recursion as two phases.



The first phase is when the function keeps calling itself.

The second phase starts after the base case is reached.

This distinction becomes extremely important when we decide **where to place our work**, such as `print()`.

5. INTUITION WITH A REAL-LIFE EXAMPLE

Imagine you are standing at the top of a staircase and want to reach the ground.

Instead of thinking about all the stairs at once, you can think:

Take one step down



Now solve the same problem from the next step



Take one step down



Repeat

Eventually you reach the ground.

5th step



4th step



3rd step



2nd step



1st step



Ground



STOP

This is similar to recursion.

The current problem says:

"Solve this problem."

The recursive call says:

"Solve the same problem, but with a smaller input."

6. HOW RECURSION USES THE CALL STACK

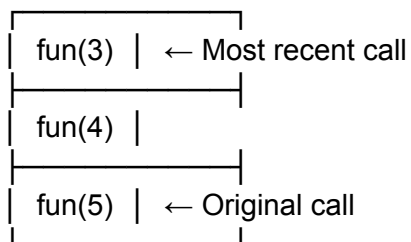
When a function calls another function, Python needs to remember where the previous function was.

Recursive calls are stored in the **call stack**.

For example:

```
fun(5)
  ↓
fun(4)
  ↓
fun(3)
```

The stack can be visualized as:



When `fun(3)` finishes, it is removed.

Then `fun(4)` continues.

Then `fun(5)` continues.

This follows:

LIFO

which means:

Last In, First Out.

7. PROBLEM 1 — PRINT N TO 1

Problem Statement

Given a number **N**, print all numbers from **N** down to **1** using recursion.

Example:

N = 5

Expected output:

5
4
3
2
1

7.1 Intuition

Start with **5**.

We need:

Print 5

Then the remaining problem is:

Print 4 to 1

So we call the same function with:

$5 - 1 = 4$

Then:

Print 4

and solve:

Print 3 to 1

This continues until:

num = 0

At that point, there is nothing left to print.

So we stop.

The idea is:

Print current number



Solve smaller problem



Print current number



Solve smaller problem



...



Stop

7.2 Code

```
def n_to_1(num):
```

```
    if num == 0:
        return
```

```
    print(num)
```

```
    n_to_1(num - 1)
```

```
n_to_1(5)
```

7.3 Understanding the Code

Step 1 — Function receives the number

```
def n_to_1(num):
```

Initially:

```
num = 5
```

Step 2 — Check the base case

```
if num == 0:  
    return
```

If `num` becomes `0`, recursion stops.

Step 3 — Print the current number

```
print(num)
```

For the first call:

```
5
```

is printed.

Step 4 — Make the problem smaller

```
n_to_1(num - 1)
```

If:

```
num = 5
```

the next call becomes:

```
n_to_1(4)
```

Then:

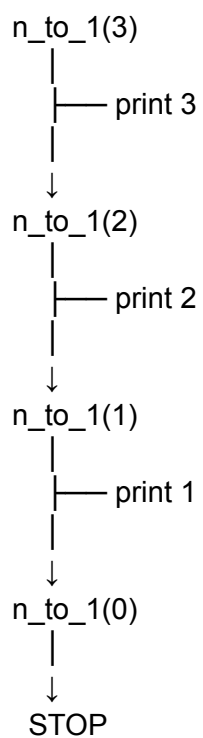
```
n_to_1(3)
n_to_1(2)
n_to_1(1)
n_to_1(0)
```

8. DRY RUN — N TO 1

For:

```
n_to_1(3)
```

The execution is:



Output:

```
3
2
1
```

Notice that printing happens **before** the recursive call.

Therefore, the output is produced while going downward.

9. PROBLEM 2 — PRINT 1 TO N

Problem Statement

Given a number N , print all numbers from 1 to N using recursion.

Example:

$N = 5$

Expected output:

1
2
3
4
5

9.1 Intuition

This time, instead of printing immediately, we first solve the smaller problem.

For:

5

we first ask:

Print 1 to 4

Then:

Print 1 to 3

Then:

Print 1 to 2

Then:

Print 1 to 1

Then:

Print 1 to 0

We reach the base case.

Now the functions start returning.

During the return:

```
print 1  
print 2  
print 3  
print 4  
print 5
```

So the output becomes ascending.

9.2 Code

```
def one_to_n(num):
```

```
    if num == 0:  
        return
```

```
    one_to_n(num - 1)
```

```
    print(num)
```

```
one_to_n(5)
```

10. DRY RUN — 1 TO N

For:

one_to_n(3)

First, recursion goes downward:

```
one_to_n(3)
  ↓
one_to_n(2)
  ↓
one_to_n(1)
  ↓
one_to_n(0)
  ↓
STOP
```

Now the functions return:

```
one_to_n(1) → print 1
one_to_n(2) → print 2
one_to_n(3) → print 3
```

Output:

```
1
2
3
```

11. THE MOST IMPORTANT DIFFERENCE

Compare the two programs.

N TO 1

```
print(num)
n_to_1(num - 1)
```

Output:

```
5
4
3
2
```

1

Because work happens **before** recursion.

```
5 ← PRINT
↓
4 ← PRINT
↓
3 ← PRINT
↓
2 ← PRINT
↓
1 ← PRINT
↓
0 ← STOP
```

1 TO N

```
one_to_n(num - 1)
print(num)
```

Output:

```
1
2
3
4
5
```

Because work happens **after** recursion.

```
5
↓
4
↓
3
↓
2
↓
1
↓
0 ← STOP
↑
PRINT 1
```

↑
PRINT 2
↑
PRINT 3
↑
PRINT 4
↑
PRINT 5

12. PROBLEM-SOLVING FRAMEWORK FOR RECURSION

Whenever you get a recursion problem, do not immediately start writing code.

First answer these questions:

Question 1

What is the smallest problem?

That usually gives you the base case.

Question 2

What should happen at the current value?

This determines the work.

Question 3

How can I reduce the problem?

For example:

$N \rightarrow N - 1$

Question 4

What happens before the recursive call?

This is the downward phase.

Question 5

What happens after the recursive call?

This is the returning/upward phase.

13. COMMON MISTAKES

Mistake 1 — No Base Case

Incorrect:

```
def fun(num):  
    print(num)  
    fun(num - 1)
```

There is no stopping condition.

Mistake 2 — Moving Away From the Base Case

Incorrect:

```
def fun(num):  
  
    if num == 0:  
        return  
  
    fun(num + 1)
```

The values become:

$5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow \dots$

The base case is never reached.

Mistake 3 — Confusing Before and After

These two are different:

```
print(num)
fun(num - 1)
```

and:

```
fun(num - 1)
print(num)
```

The first produces:

$N \rightarrow 1$

The second produces:

$1 \rightarrow N$

14. KEY THINGS TO REMEMBER

14.1 The Three Essentials

Every recursion problem needs:

Base Case

Recursive Call

Progress Toward Base Case

14.2 Base Case

Remember:

The base case tells recursion when to stop.

Example:

```
if num == 0:  
    return
```

14.3 Recursive Call

Remember:

The recursive call solves a smaller version of the same problem.

Example:

```
fun(num - 1)
```

14.4 Progress

Remember:

Every recursive call must move toward the base case.

Example:

$5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

14.5 Before vs After

The most important printing pattern:

Work $\rightarrow$ Recursion

means work happens while going down.

Recursion $\rightarrow$ Work

means work happens while coming back up.

14.6 Call Stack

Remember:

Every recursive call creates a new stack frame.

The stack follows:

LIFO

Last In, First Out.

15. COMPLEXITY

For:

```
def n_to_1(num):
```

```
    if num == 0:  
        return
```

```
    print(num)  
    n_to_1(num - 1)
```

If `num = N`, approximately `N` recursive calls are made.

Therefore:

Time Complexity = $O(N)$

Because the call stack stores the recursive calls:

Space Complexity = $O(N)$

The space is not from the output. It comes from the **recursion call stack**.

16. QUICK REVISION

RECURSION

↓

Function calls itself

↓
Problem becomes smaller
↓
Eventually reaches base case
↓
Recursion stops
↓
Functions return

Core pattern:

```
def fun(num):  
  
    if base_case:  
        return  
  
    # work  
  
    fun(smaller_problem)
```

Print N → 1:

```
print(num)  
fun(num - 1)
```

Print 1 → N:

```
fun(num - 1)  
print(num)
```

Remember:

Base Case → When to stop
Progress → How to reach the stop
Recursive Call → Solve smaller problem
Call Stack → Keeps track of unfinished calls
Before recursion → Downward phase
After recursion → Upward/returning phase

17. PRACTICE QUESTIONS

Solve these in order:

1. Print your name **N** times using recursion.
2. Print **N** to **1**.
3. Print **1** to **N**.
4. Write a function that prints something before and after the recursive call. Predict the output before running it.
5. Find the sum of numbers from **1** to **N**.
6. Find the factorial of **N**.
7. Reverse an array using recursion.
8. Check whether a string is a palindrome using recursion.

For every problem, first identify:

1. Base case
2. Recursive call
3. Progress
4. Work before recursion
5. Work after recursion

That should be your standard approach whenever you encounter a new recursion problem.